Physical Design for Nanometer ICs
Programming Assignment #1
2-Way F-M Circuit Partitioning

臺科大 B11107051 李品翰

2026-03-31

1 Partition 主要程式

本階段負責讀取測資、初始化資料，並作為進入多層次分割 (Multilevel Partitioning) 的入口。

Algorithm 1 Main Partition Flow

```
初始化所有 cell 的 size 為 1
呼叫 hMetis 進行多層次分割
取得最終的 cut size 與 cell 分組結果並輸出
```

1.1 棄案：重複執行 hMetis

在初期的設計中，原本打算利用剩餘的執行時間，加入 Multi-start 的機制重複執行多次完整的 hMetis 流程以尋求最佳解。但經過實驗後發現，最終的 cut size 降低幅度並不明顯，而 runtime 卻直接成倍增長。考量到計分方式中 runtime 佔了 20% 的權重，為了一點點的 cut size 犧牲大量 runtime 並不划算，因此最終決定主程式只呼叫一次 hMetis。

2 hMetis 多層次分割流程

hMetis 結合了 Clustering 與 F-M Partitioning，透過遞迴的方式將龐大的電路圖逐步縮小，求解後再逐步還原微調。

Algorithm 2 hMetis Recursive Flow

```
執行一次 iteration 的 pre-partition (FM)
if pre-partition 失敗 (無法滿足平衡) then
    return 失敗
end if
執行 First Choice Clustering 將圖縮小
遞迴呼叫 hMetis
if 下一層 hMetis 回傳失敗 then
    執行 n 次完整的 F-M Partitioning 取最佳解 (Initial Partitioning)
else
    將下一層的結果解開 (Declustering)
    執行 F-M Partitioning 進行微調 (Refinement)
end if
return 本層的 partition 結果
```

2.1 棄案：固定的遞迴停止條件 (Threshold)

關於 hMetis 的終止條件，一開始是設定當 cell 數量低於一個固定的 threshold 時就停止 clustering 並開始 Initial Partitioning。但後來發現固定 threshold 會造成兩個問題：

1. 大型測資在多次 clustering 後，容易出現體積過大的超級節點 (Cluster)，導致後續的 partition 無法找到合乎 balance constraint 的解而失敗。
2. 小型測資可能因為初始 cell 數量就低於 threshold，導致一次 clustering 都沒做到就直接進入 partition。

基於以上狀況，最後決定讓程式自己動態尋找 hMetis 的深度：持續遞迴進行 clustering，直到下一層的 partition 因為 cluster 太大找不到平衡解而回傳失敗時，才停止遞迴並以此深度作為最底層開始反向 Refinement。

3 First Choice Clustering

本階段透過 First Choice 演算法，將連結緊密的 cell 綁定成超級節點，以減少圖的規模並保留自然的群集特徵。主要邏輯與講義概念一致。

Algorithm 3 First Choice Clustering

```
for 每一個未被綁定的 cell do
    尋找與其連結的最高 weight 目標 (若平手則選擇 size 較小者)
    將該 cell 併入目標 cluster 中，並更新 cluster size
end for
建立新的 net 與 cluster 映射表，回傳縮小後的圖
```

3.1 略過巨大 Net 的優化

實作上唯一比較特別的處理是「跳過 cell 數量太多的 net」。因為連接極多 cell 的超級大 net 對於 weight 的計算貢獻值極低 $(1/(|e|-1))$，且掃描它們會耗費大量時間，因此在計算權重時直接忽略這些巨型 net，可以大幅加快 clustering 的速度且幾乎不影響最終品質。

4 F-M Partitioning

此為最核心的演算法，採用 $O(1)$ 的 Bucket List 資料結構進行節點搬移與 Gain 更新，邏輯同樣基於標準的 F-M 演算法。

Algorithm 4 Fiduccia-Mattheyses Partitioning

```
repeat
    紀錄本回合初始的 cut size 為 best_cut_size
    初始化狀態（包含解除所有 cell 的鎖定）
    計算所有 cell 的 initial gain 並建立 Bucket List
    while 還有未鎖定的 cell 且尚未超時 do
        根據策略選出最佳的 cell 進行移動
        將該 cell 移至對立 group 並鎖定
        更新相鄰 cell 的 gain
        檢查並紀錄過程中的 min cut size (需滿足平衡條件)
    end while
    Trace back 回到 min cut size 發生的狀態
until Trace back 後的 cut size ≥ best_cut_size
```

4.1 Cell 選擇策略 (Tie-breaking)

當需要選擇要移動的 cell 時，若兩個 group 的 top cell 都滿足 balance 條件，則依照以下順位進行選擇：

1. Gain 較大者優先。
2. 選擇所在 Group size 較大的一邊，以盡量維持兩邊的平衡。

這樣可以避免未來演算法為了滿足 balance constraint，而被迫選擇 gain 較差的 cell。

4.2 非法狀態處理

在 F-M 的過程中，搬移 cell 經常會遇到破壞平衡 (balance constraint) 的情況，這主要基於兩個原因：

1. 若 balance degree 設定極小且為偶數個 cell，搬動 1 個 cell 就會違反平衡，必須連續搬動 2 個才能恢復平衡。

2. 當某個 cell (即 Cluster) 的 size 太大時，移動它雖然能有效降低 gain，但會立刻打破平衡限制。

為了解決這個問題，程式允許這些破壞平衡的 move 發生，但只要移動後的狀態違反 balance，無論當下的 cut size 是否為歷史新低，都不會將其紀錄為 `min_cut_size`。這保證了 iteration 結束後的 trace back 絕對不會回到 invalid 的狀態。若整個 iteration 跑完都沒有出現過任何一次合法的分組，則判定此次 partition 失敗。